

AWS VPC SETUP

Prepared By: kritika Agarwal

Project: AWS VPC Setup and VPC Peering

Objective

- Login AWS
- Purpose

Architecture Diagram

- Picture
- Structure

Resource Created

- VPC Creation
- SUBNET Creation

Configuration Details

- Internet Gateway
- Attach IGW
- NAT Gateway

Security Rules

- Security Group
- EC2 Launch

Route Tables

- Public
- Private

Testing Results

- Testing
- Peering

Conclusion

- Result
- Final summary

1. OBJECTIVE -

- Purpose: To create a secure AWS VPC with Public and Private Subnets and configure networking components.

Steps:

1. Login to AWS Management Console.
2. Search VPC in the search bar.
3. Open the VPC Dashboard.

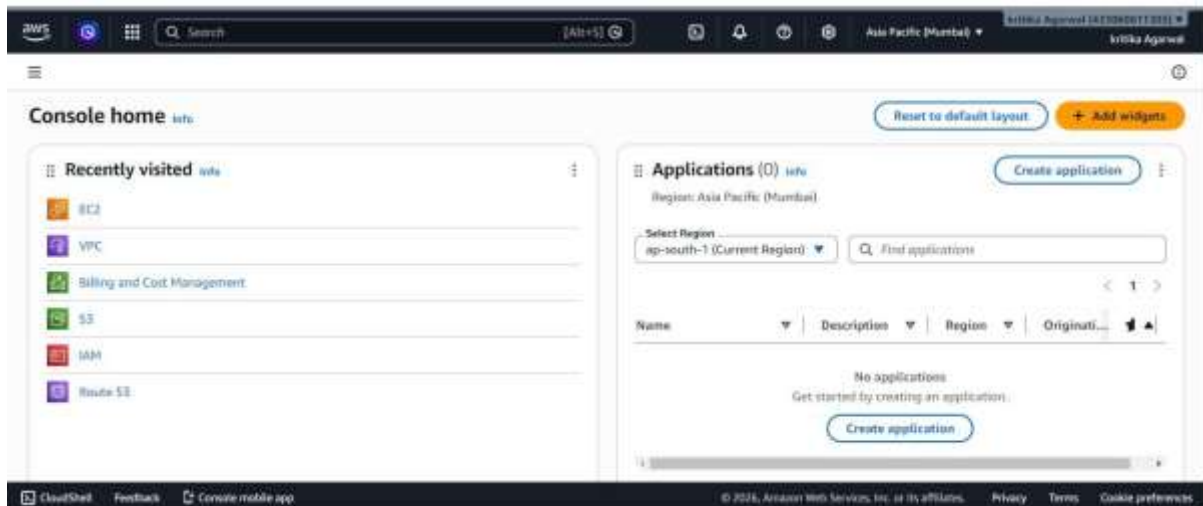


FIG-1: AWS CONSOLE HOME PAGE

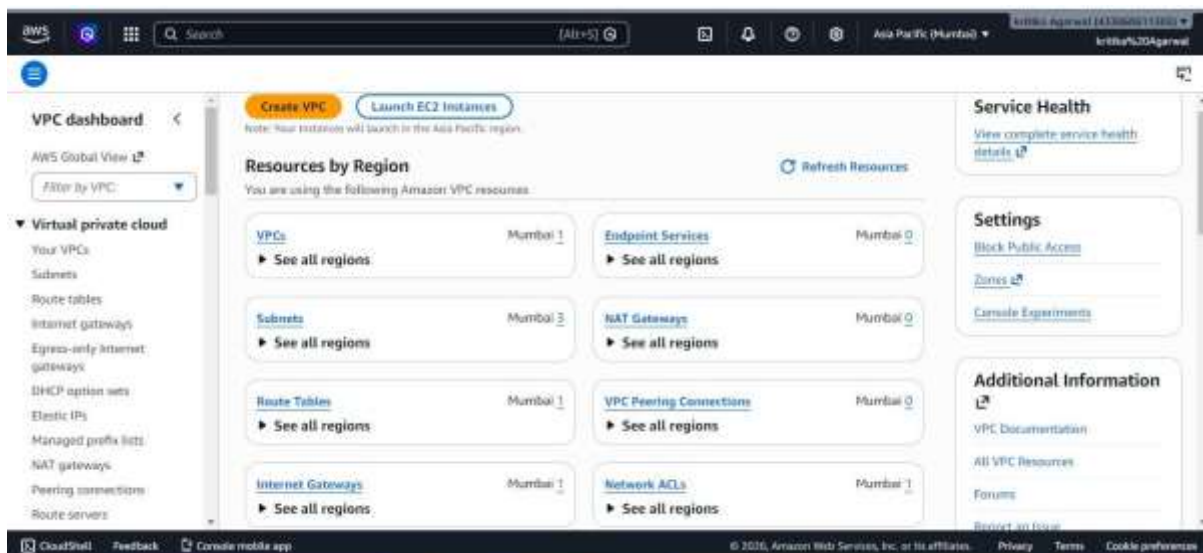


FIG-2: VPC DASHBOARD

2. ARCHITECTURE DIAGRAM-

The architecture diagram represents the complete AWS VPC network. It shows the VPC, Public Subnet, Private Subnet, Internet Gateway, NAT Gateway, Route Tables, Security Groups, and EC2 instances. This diagram helps to understand how all AWS networking components are connected.

Description: This architecture contains-

- VPC
- Public Subnet
- Private Subnet
- Internet Gateway
- NAT Gateway
- Route Table
- EC2 Instances
- Subnet Association
- VPC Peering

FIG-1: ARCHITECTURE DIAGRAM-

AWS VPC ARCHITECTURE (PUBLIC SUBNET):

- Created a VPC with CIDR block 170.32.0.0/16 in the Mumbai region.
- Attached an Internet Gateway (IG) to provide internet access.
- Configured a Route Table with internet routing.
- Associated the Route Table with the Public Subnet.
- Launched an EC2 instance (Virtual Machine) in the Public Subnet.
- Users can access the EC2 instance through the Internet using the Internet Gateway.

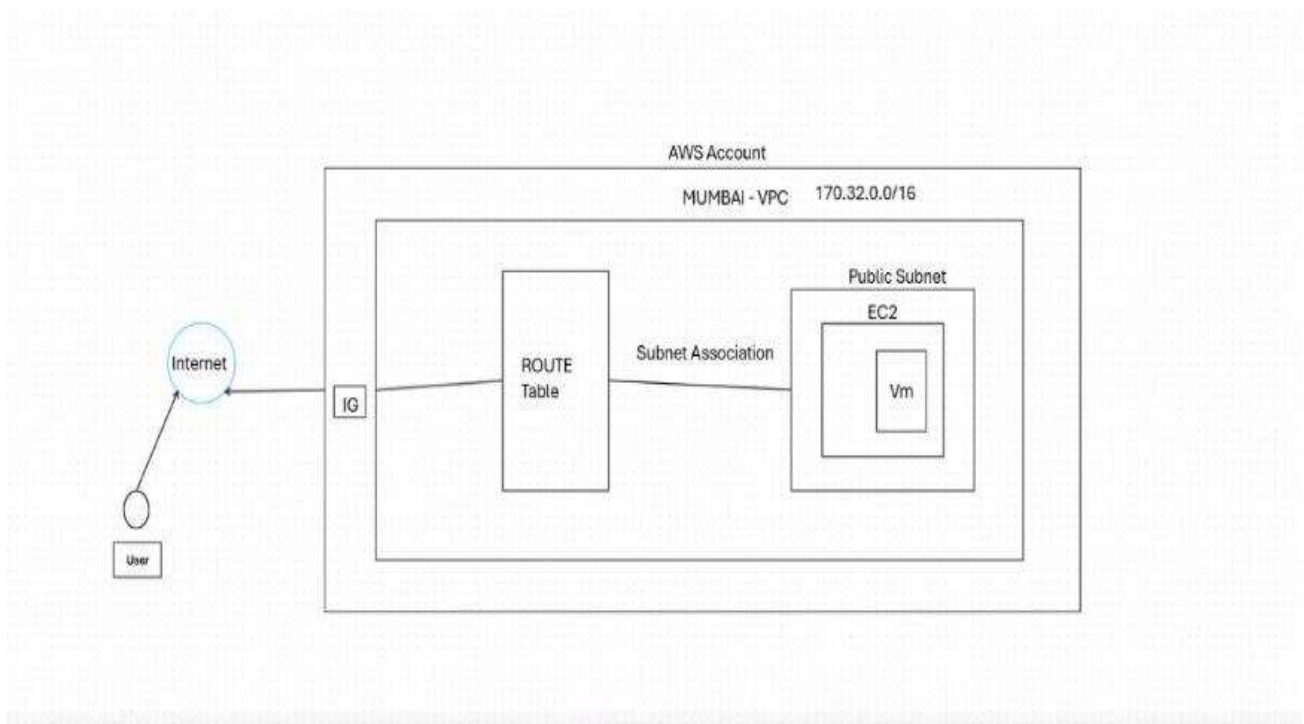


FIG-2: ARCHITECTURE DIAGRAM-

AWS VPC ARCHITECTURE (PUBLIC & PRIVATE SUBNET):

- Created a VPC in the Mumbai Region.
- Configured one Public Subnet and one Private Subnet.
- Attached an Internet Gateway (IG) to the VPC.
- Created separate Route Tables and associated them with each subnet.
- Launched EC2 instances in both Public and Private Subnets.
- The Public EC2 can access the internet, while the Private EC2 remains isolated and is used for secure internal communication.

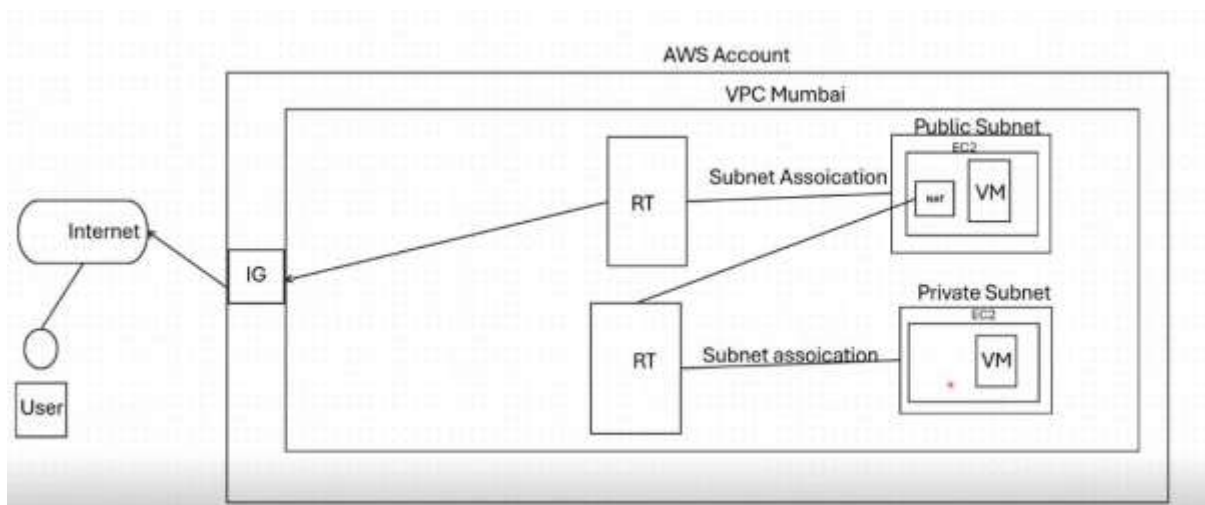
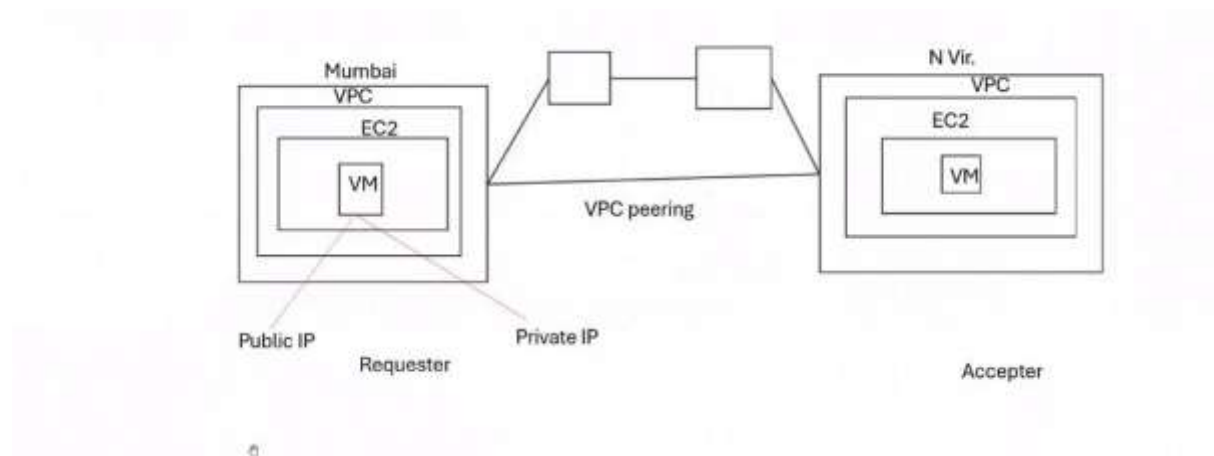


FIG-3: ARCHITECTURE DIAGRAM-

AWS VPC PEERING ARCHITECTURE :

- Created two VPCs in different AWS regions.
- Launched an EC2 instance in each VPC.
- Established a VPC Peering Connection between the VPCs.
- One VPC acts as the Requester and the other as the Acceptor.
- Updated Route Tables to enable communication.
- The EC2 instances communicate securely using Private IP addresses without using the public internet.



3. RESOURCE CREATED-

Step 1: Create VPC:

- Open VPC Dashboard.
- Click Create VPC.
- Enter:
 1. Name: my-vpc-01
 2. CIDR: 170.32.0.0/16
- Region
- Click Create VPC

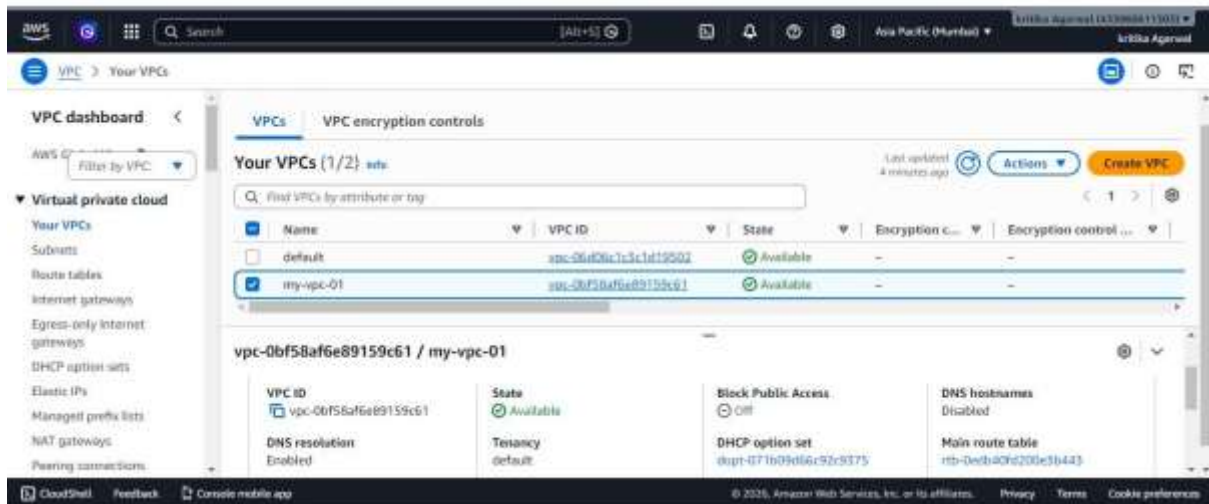


FIG: MY-VPC

Step 2: Create subnet:

- Click Create Subnet
- Enter:
 1. VPC ID
 2. Name: my-subnet-01
 3. Availability Zone: ap-south-1a
 4. IPv4 VPC CIDR Block: 170.32.0.0/16
 5. IPv4 Subnet CIDR Block: 170.32.0.0/24
- Create Subnet

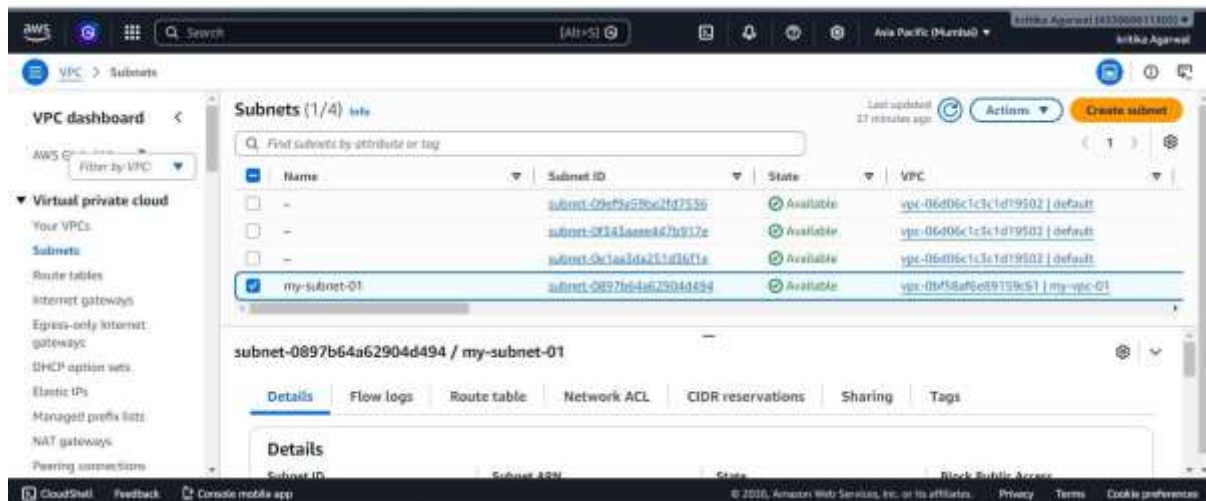


FIG: MY-SUBNET

Step 3: Create Public Subnet:

- Click Subnets

- Click Create Subnet
- Select your VPC
- Name: my-vpc-01-public
- CIDR: 170.32.0.0/16
- Select Availability Zone.
- Click Create

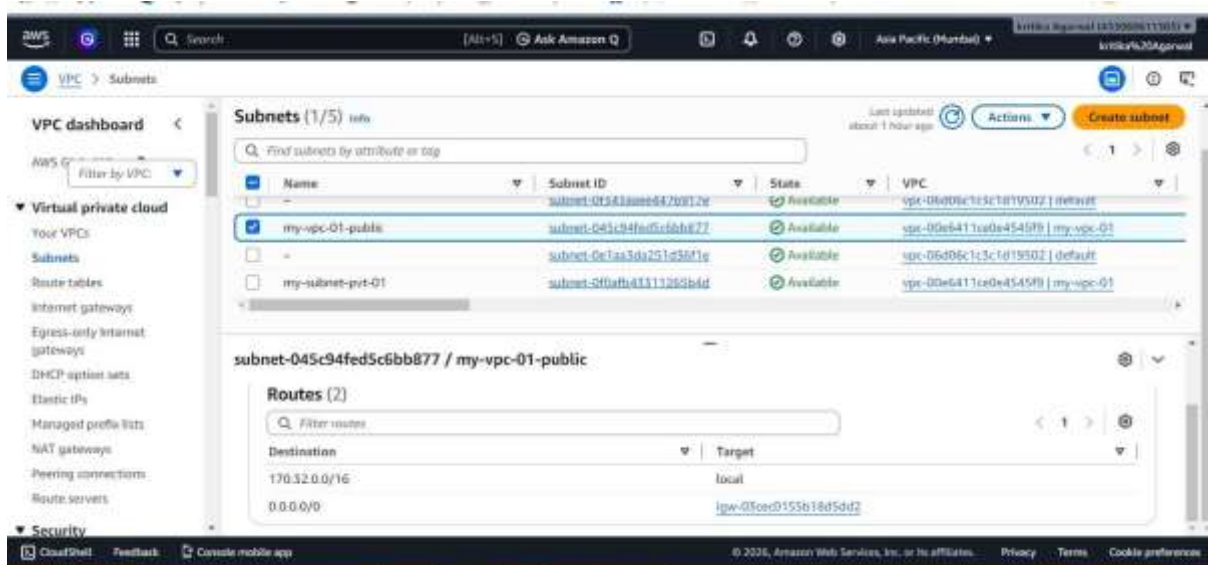


FIG:PUBLIC SUBNET

Step 4: Create Private Subnet:

- Click Create Subnet
- Select your VPC
- Name: my-subnet-pvt-01
- CIDR: 170.32.0.0/16
- Select Availability Zone
- Click Create

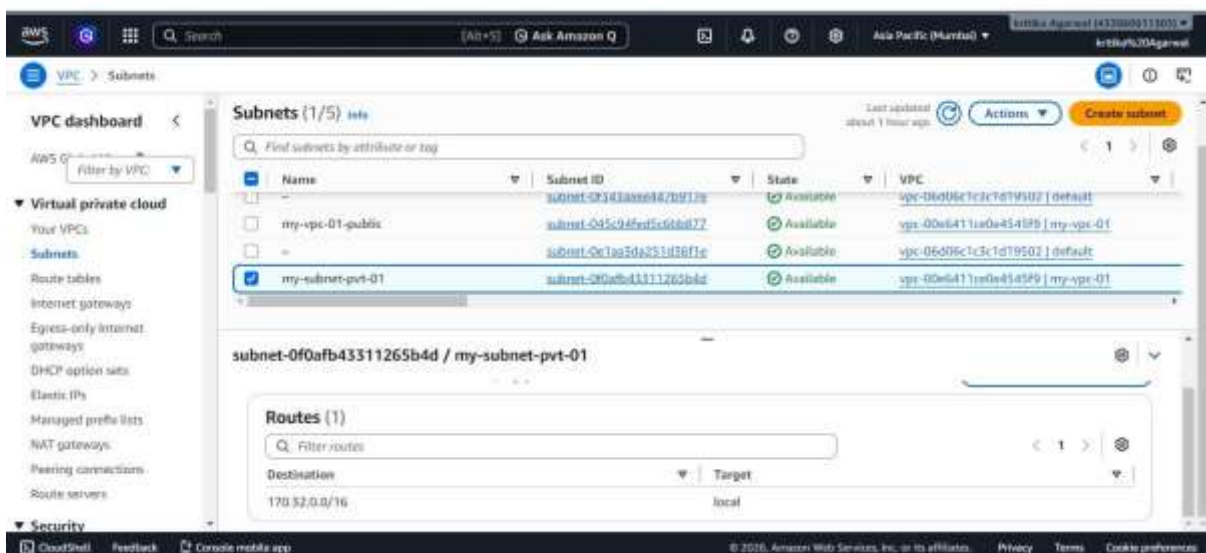


FIG: PRIVATE SUBNET

4. CONFIGURATION DETAILS-

a. Attach Internet Gateway:

- Click Internet Gateway
- Click Create Internet Gateway
- Enter Name
- Click Create
- Click Attach to VPC
- Select your VPC

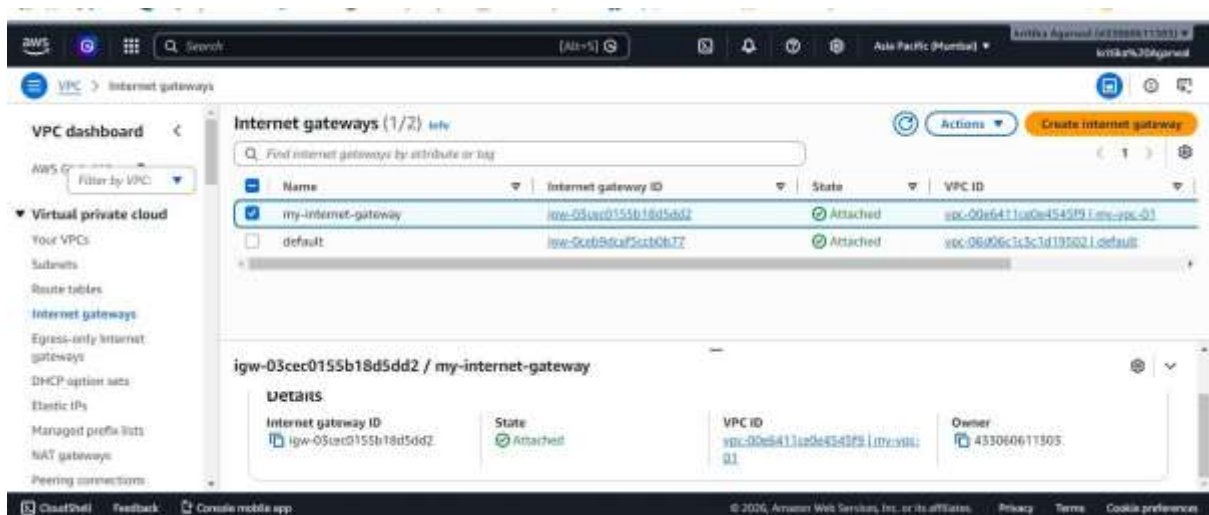


FIG: ATTACH INTERNET GATEWAY

b. NAT GATEWAY:

- Open the VPC Dashboard.
- Click NAT Gateways.
- Click Create NAT Gateway.
- Enter-
 - (1)Name: my-nat-gateway
 - (2)Subnet: Public Subnet
 - (3)Connectivity Type: Public
 - (4)Elastic IP: Allocate a new Elastic IP
- Update Private Route Table-
 - (1)Destination:0.0.0.0/0
 - (2)Target: NAT Gateway

- Save the route and test internet access from the private EC2 instance.

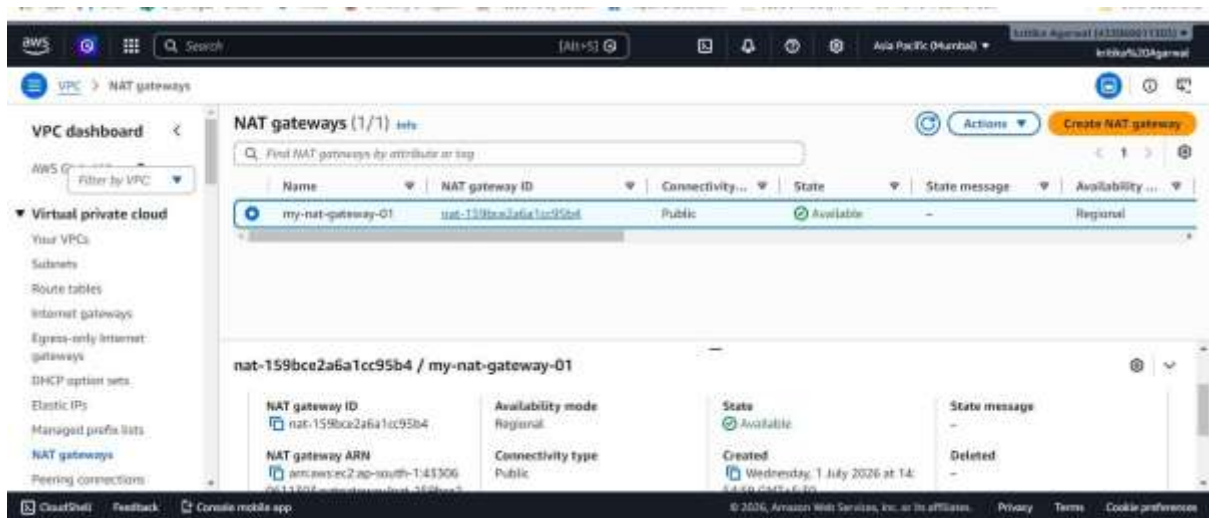


FIG: NAT GATEWAY

5. SECURITY RULES:

❖ SECURITY GROUPS-

A Security Group acts as a virtual firewall for an EC2 instance. It controls the inbound (incoming) and outbound (outgoing) traffic. In this project, the Security Group was configured to allow SSH, ICMP (Ping), HTTP, and HTTPS traffic.

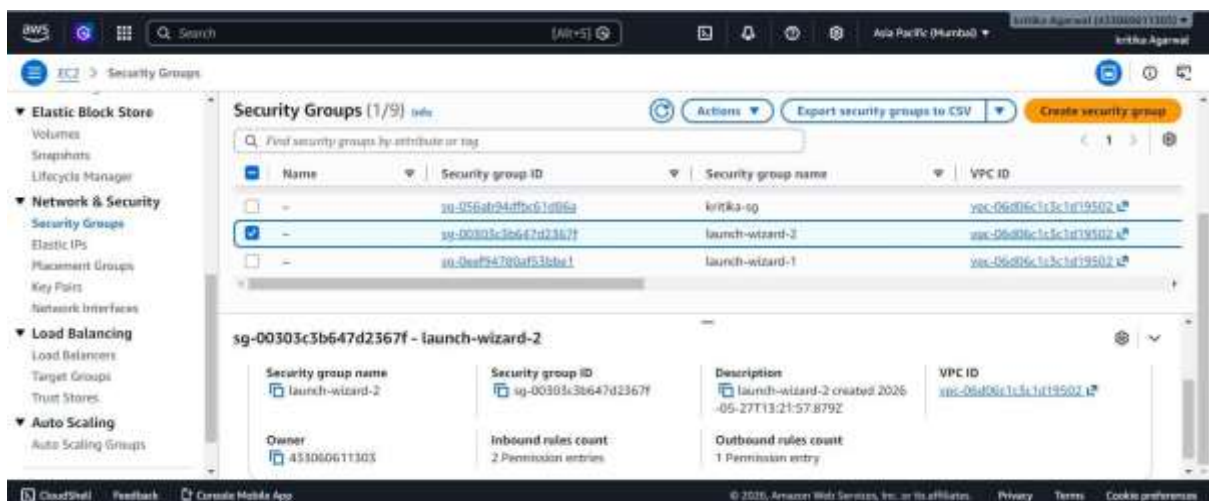


FIG: SECURITY GROUP

1. INBOUND RULES-

Type	Protocol	port	Source	Purpose
SSH	TCP	22	My IP	Remote login to EC2 instance.
ICMP (All ICMP-IPv4)	ICMP	N/A	Anywhere(0.0.0.0/0) Or VPC CIDR	Allow Ping requests.
HTTP	TCP	80	0.0.0.0/0	Allow web traffic.
HTTPS	TCP	443	0.0.0.0/0	Allow secure web traffic.

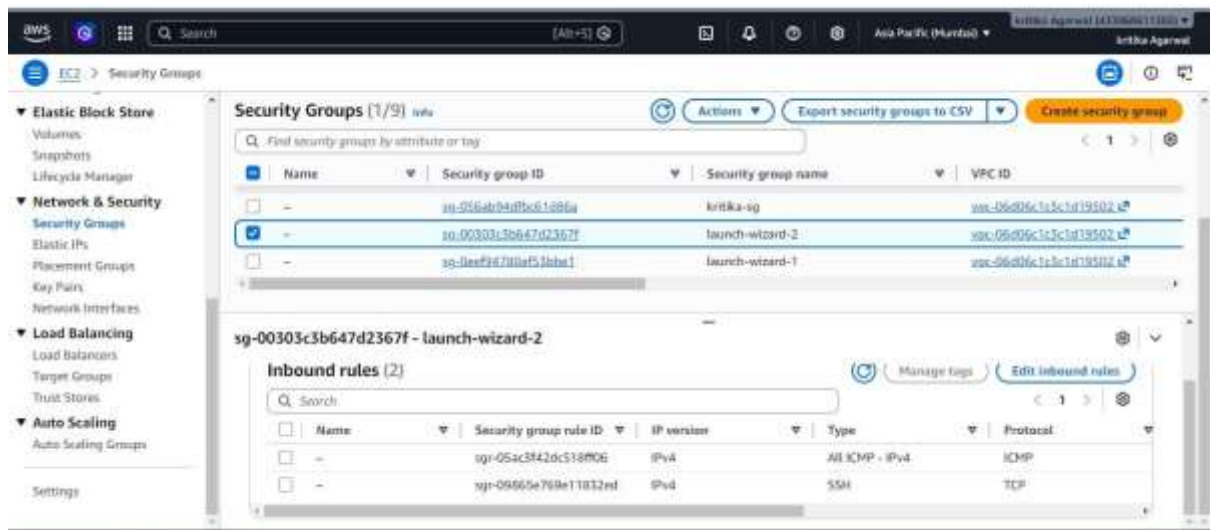


FIG: INBOUND RULES

2. OUTBOUND RULES-

Type	Protocol	Port	Source	purpose
All Traffic	All	All	0.0.0.0/0	Allow outbound internet access.

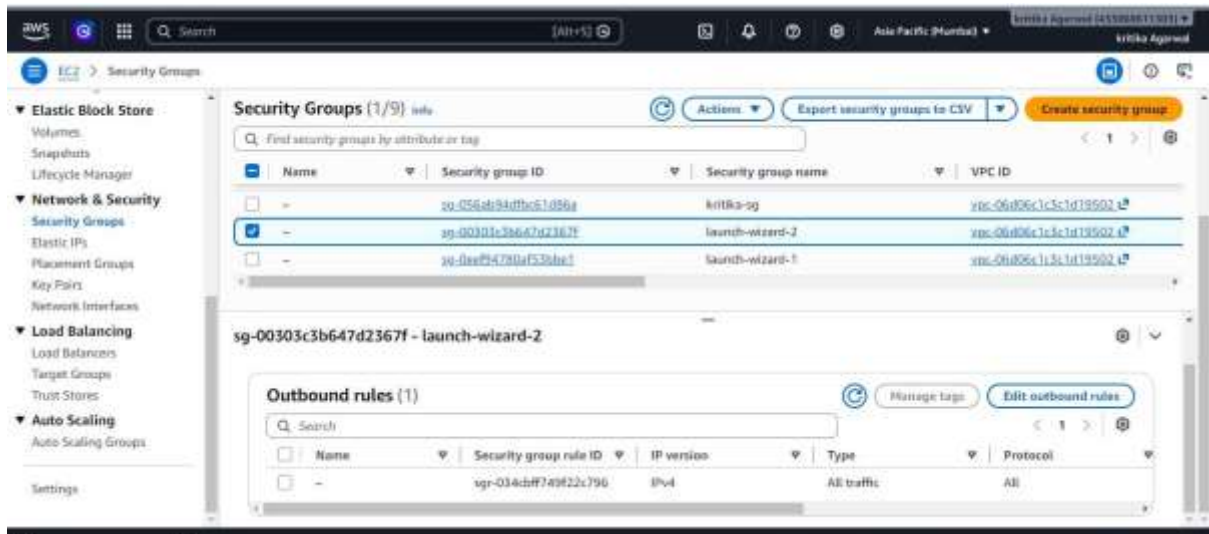
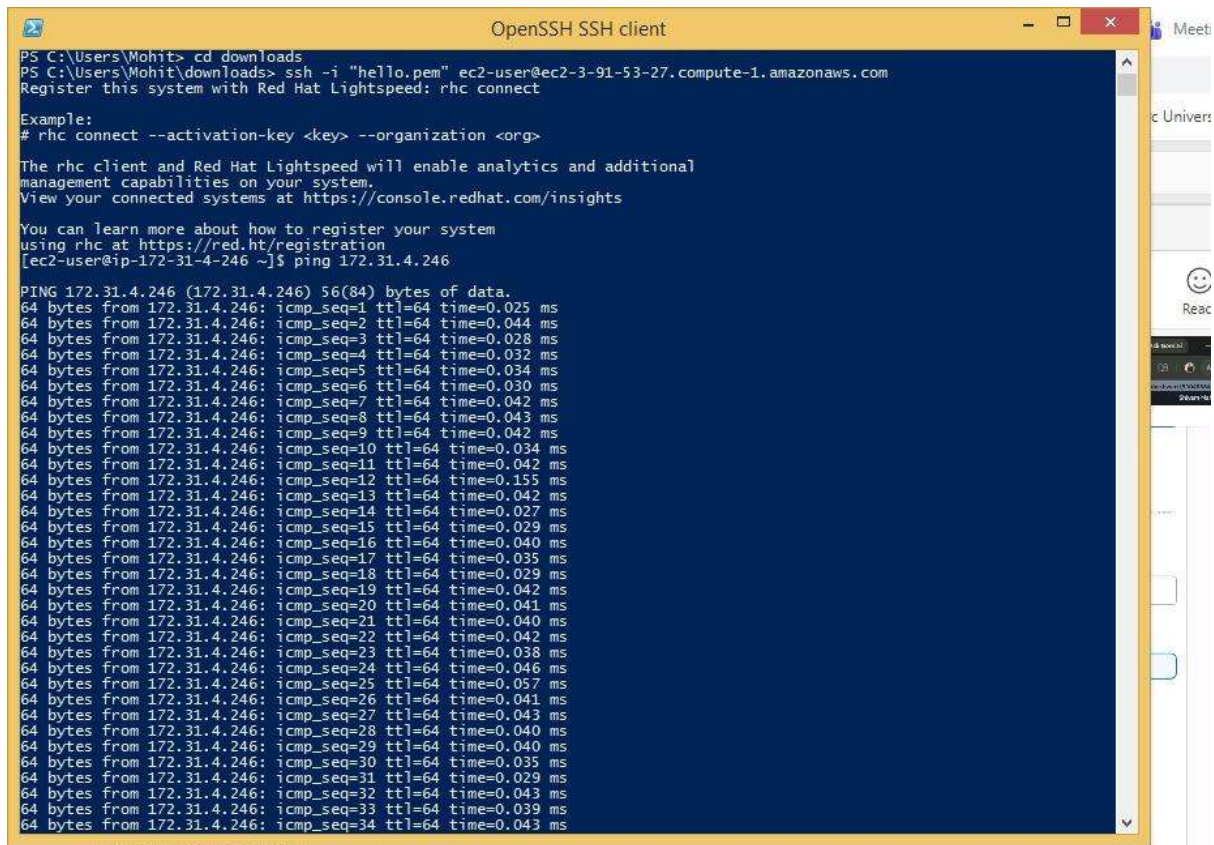


FIG: OUTBOUND RULES

3. ICMP(PING)- (Internet Control Message Protocol) is used to test network connectivity. The ping command uses ICMP packets to check whether an EC2 instance is reachable.

EXAMPLE:ping<EC2-IP-Address>



The screenshot shows a terminal window titled "OpenSSH SSH client". The user is logged in as "ec2-user@ec2-3-91-53-27.compute-1.amazonaws.com". The terminal displays the following commands and output:

```
PS C:\Users\Mohit> cd downloads
PS C:\Users\Mohit\downloads> ssh -i "hello.pem" ec2-user@ec2-3-91-53-27.compute-1.amazonaws.com
Register this system with Red Hat Lightspeed: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Lightspeed will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
[ec2-user@ip-172-31-4-246 ~]$ ping 172.31.4.246

PING 172.31.4.246 (172.31.4.246) 56(84) bytes of data:
64 bytes from 172.31.4.246: icmp_seq=1 ttl=64 time=0.025 ms
64 bytes from 172.31.4.246: icmp_seq=2 ttl=64 time=0.044 ms
64 bytes from 172.31.4.246: icmp_seq=3 ttl=64 time=0.028 ms
64 bytes from 172.31.4.246: icmp_seq=4 ttl=64 time=0.032 ms
64 bytes from 172.31.4.246: icmp_seq=5 ttl=64 time=0.034 ms
64 bytes from 172.31.4.246: icmp_seq=6 ttl=64 time=0.030 ms
64 bytes from 172.31.4.246: icmp_seq=7 ttl=64 time=0.042 ms
64 bytes from 172.31.4.246: icmp_seq=8 ttl=64 time=0.043 ms
64 bytes from 172.31.4.246: icmp_seq=9 ttl=64 time=0.042 ms
64 bytes from 172.31.4.246: icmp_seq=10 ttl=64 time=0.034 ms
64 bytes from 172.31.4.246: icmp_seq=11 ttl=64 time=0.042 ms
64 bytes from 172.31.4.246: icmp_seq=12 ttl=64 time=0.155 ms
64 bytes from 172.31.4.246: icmp_seq=13 ttl=64 time=0.042 ms
64 bytes from 172.31.4.246: icmp_seq=14 ttl=64 time=0.027 ms
64 bytes from 172.31.4.246: icmp_seq=15 ttl=64 time=0.029 ms
64 bytes from 172.31.4.246: icmp_seq=16 ttl=64 time=0.040 ms
64 bytes from 172.31.4.246: icmp_seq=17 ttl=64 time=0.035 ms
64 bytes from 172.31.4.246: icmp_seq=18 ttl=64 time=0.029 ms
64 bytes from 172.31.4.246: icmp_seq=19 ttl=64 time=0.042 ms
64 bytes from 172.31.4.246: icmp_seq=20 ttl=64 time=0.041 ms
64 bytes from 172.31.4.246: icmp_seq=21 ttl=64 time=0.040 ms
64 bytes from 172.31.4.246: icmp_seq=22 ttl=64 time=0.042 ms
64 bytes from 172.31.4.246: icmp_seq=23 ttl=64 time=0.038 ms
64 bytes from 172.31.4.246: icmp_seq=24 ttl=64 time=0.046 ms
64 bytes from 172.31.4.246: icmp_seq=25 ttl=64 time=0.057 ms
64 bytes from 172.31.4.246: icmp_seq=26 ttl=64 time=0.041 ms
64 bytes from 172.31.4.246: icmp_seq=27 ttl=64 time=0.043 ms
64 bytes from 172.31.4.246: icmp_seq=28 ttl=64 time=0.040 ms
64 bytes from 172.31.4.246: icmp_seq=29 ttl=64 time=0.040 ms
64 bytes from 172.31.4.246: icmp_seq=30 ttl=64 time=0.035 ms
64 bytes from 172.31.4.246: icmp_seq=31 ttl=64 time=0.029 ms
64 bytes from 172.31.4.246: icmp_seq=32 ttl=64 time=0.043 ms
64 bytes from 172.31.4.246: icmp_seq=33 ttl=64 time=0.039 ms
64 bytes from 172.31.4.246: icmp_seq=34 ttl=64 time=0.043 ms
```

FIG-PRIVATE-PEERING

❖ LAUNCH EC2 INSTANCE-

a. PUBLIC EC2 INSTANCE:

- INSTANCE NAME: Test-Public Subnet
- AMI: Red Hat Enterprise
- INSTANCE TYPE: t3.micro
- KEY PAIR: Existing/New key pair

(i) NETWORK SETTING:

- VPC: Selected Project VPC
- SUBNET: Public Subnet
- AUTO-ASSIGN PUBLIC IP: Enabled
- SECURITY GROUP:
 - SSH (Port 22)- Allowed from My IP
 - ICMP- Allowed (for ping testing)

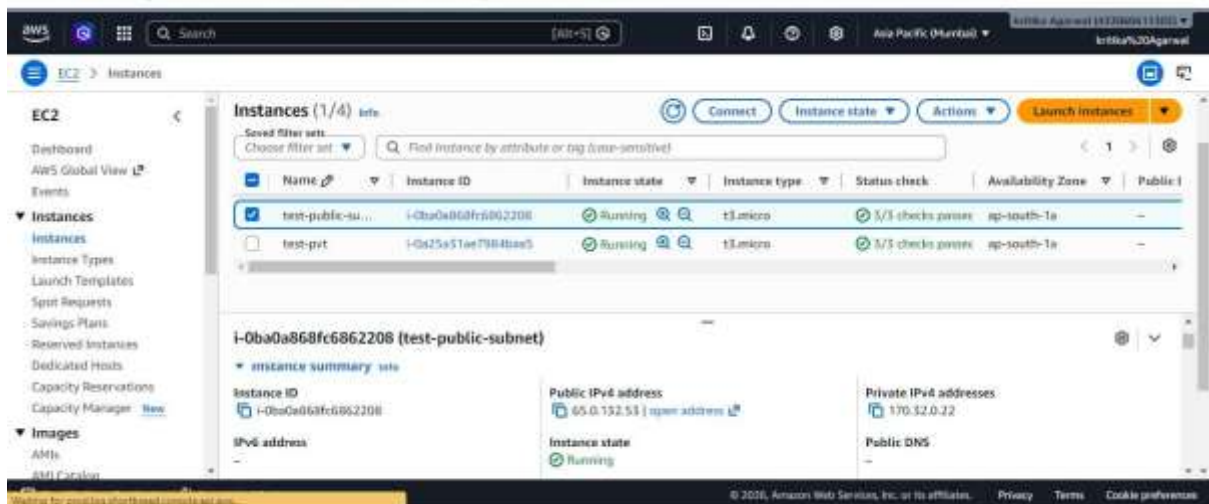


FIG-1: PUBLIC INSTANCE

b. PRIVATE EC2 INSTANCE:

- INSTANCE NAME: Test-Pvt
- AMI: Red Hat Enterprise
- INSTANCE TYPE: t3.micro
- KEY PAIR: Same Key Pair as Public EC2

(i) NETWORK SETTING:

- VPC: Selected Project VPC
- SUBNET: Private Subnet
- AUTO-ASSIGN PUBLIC IP: Disabled
- SECURITY GROUP:
 - SSH (Port 22)- Allowed from the Public EC2 Security Group or VPC CIDR.
 - ICMP- Allowed (for ping testing)

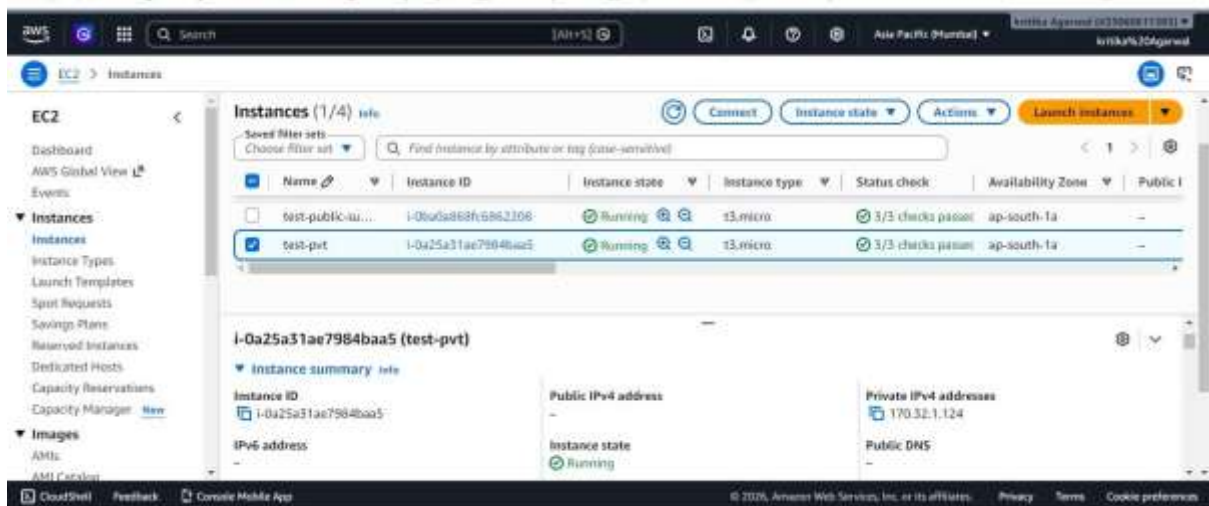


FIG-2: PRIVATE INSTANCE

6. ROUTE TABLE-

(i)- PUBLIC ROUTE TABLE:

CONFIGURATION-

- Associated with the Public Subnet.
- Contains a route for the VPC CIDR (local route).
- Default route (0.0.0.0/0) points to the Internet Gateway (IGW).

PURPOSE-

- Allows EC2 instances with a public IP to communicate with the internet.
- Supports downloading updates and packages.

ROUTE ENTRIES-

Destination	Target
170.32.0.0/16	Local
0.0.0.0/0	Internet Gateway(IGW)

DIAGRAM-

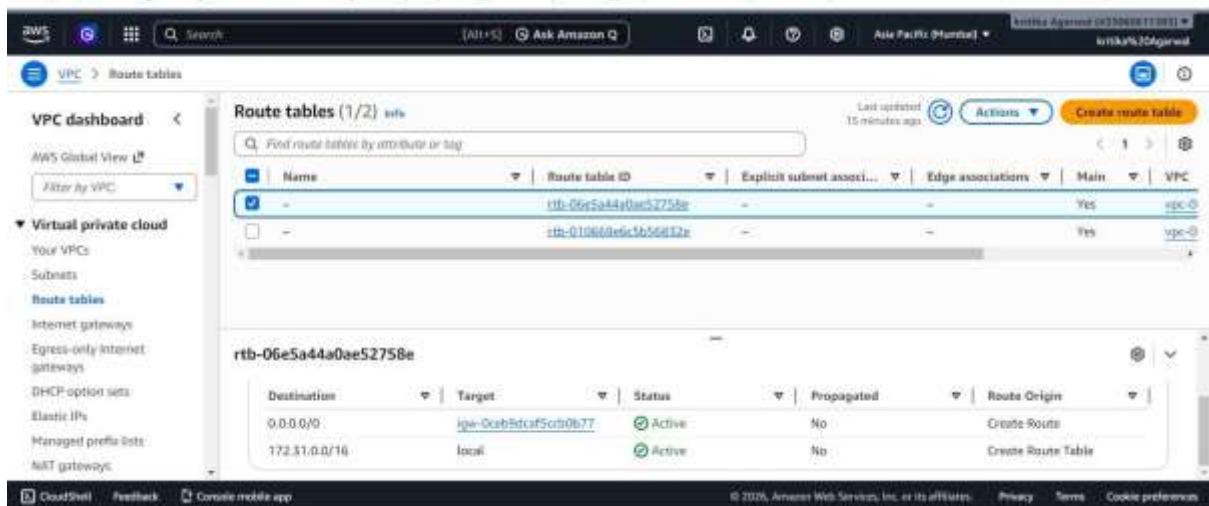


FIG: PUBLIC ROUTE TABLE

(ii)- PRIVATE ROUTE TABLE:

CONFIGURATION-

- Associated with the Private Subnet.
- Includes a route to the peer VPC CIDR through the VPC Peering Connection.
- No direct route to the Internet Gateway. If internet access is required, traffic is routed through a NAT Gateway.

PURPOSE-

- Enables communication between private subnets of two VPCs through VPC Peering.
- Allows resources in both VPCs to communicate using their private IP addresses.

ROUTE ENTRIES-

Destination	Target	purpose
VPC CIDR (10.0.0.0/16)	Local	Internal communication within the same VPC.
Peer VPC CIDR	VPC Peering connection (pcx-xxxxxxxxxxxxxxxxxx)	Communication with the peer VPC.
0.0.0.0/0	NAT Gateway	Outbound internet access without exposing the instance.

DIAGRAM-

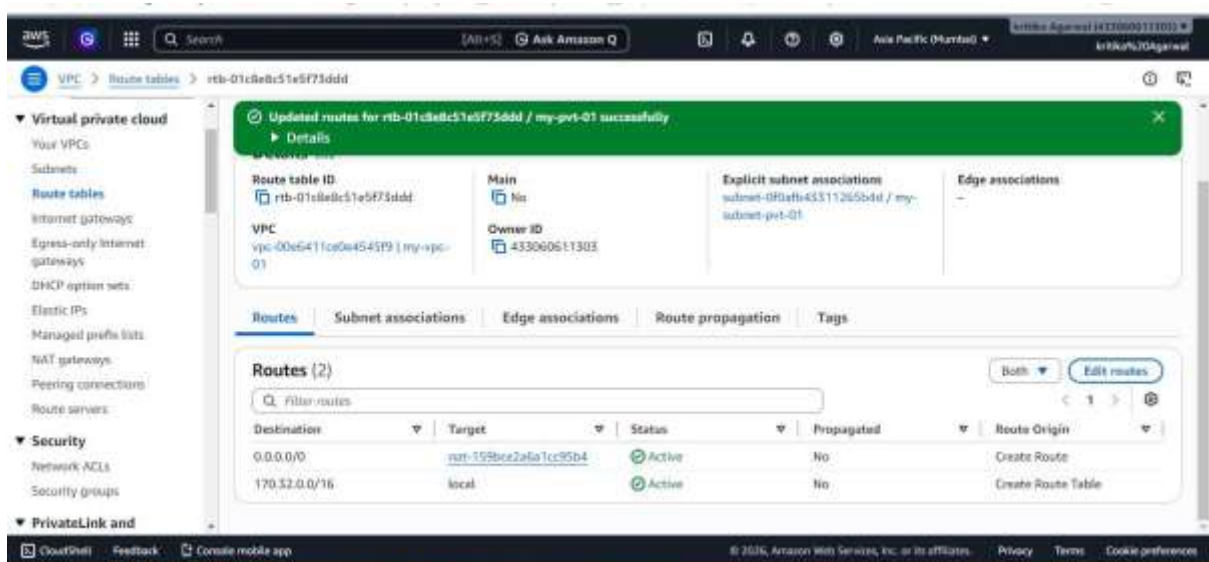


FIG: PRT-1 (PRIVATE ROUTE TABLE)

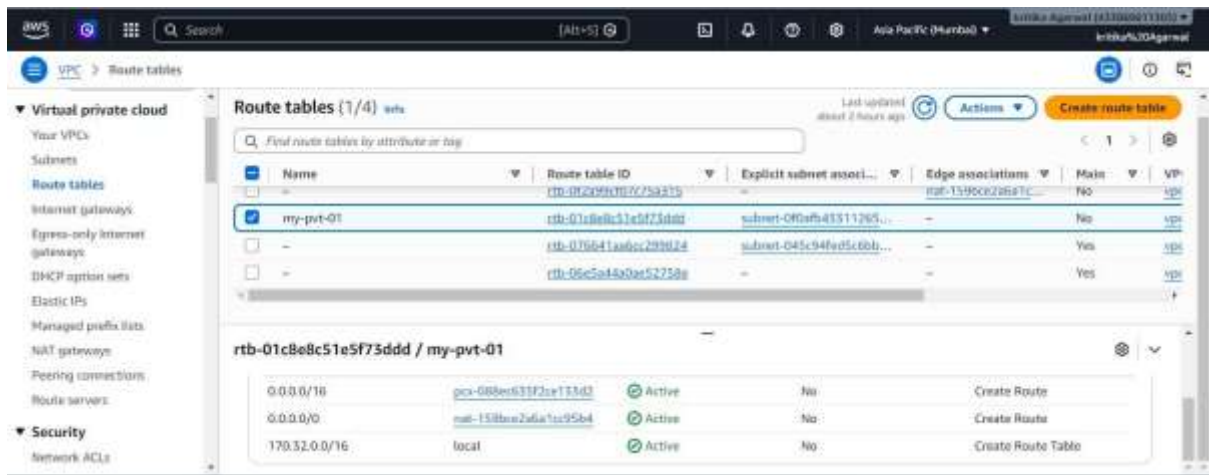


FIG: PRT-2 (ADDING VPC PEERING)

7. TESTING RESULT

- SSH Login Successful
- Internet working
- Ping Successful
- Package Installation Successful

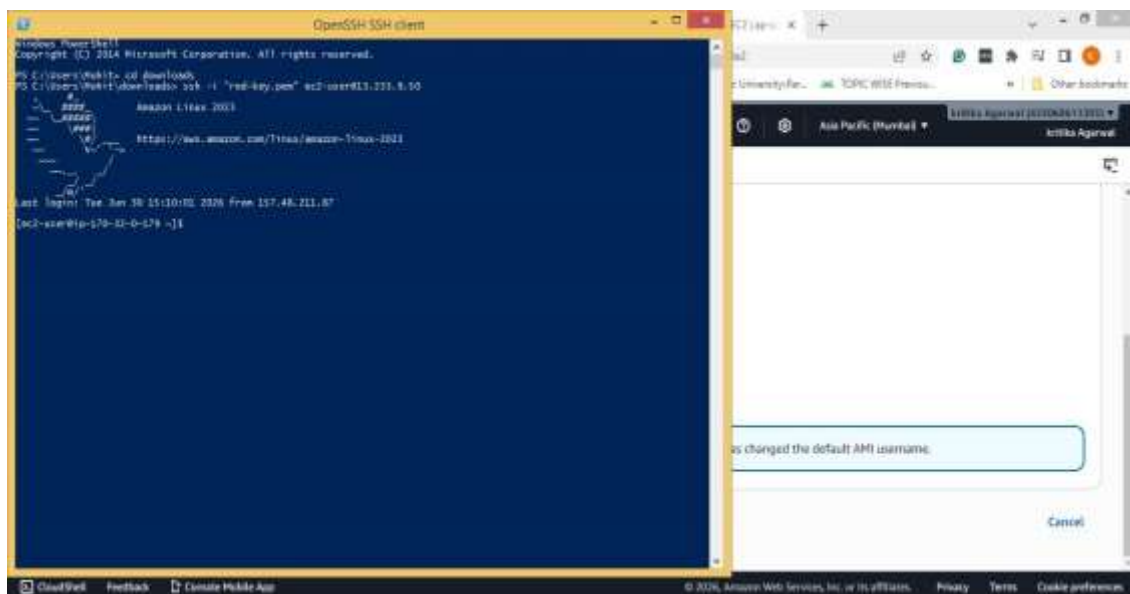


FIG: SSH CONNECTION

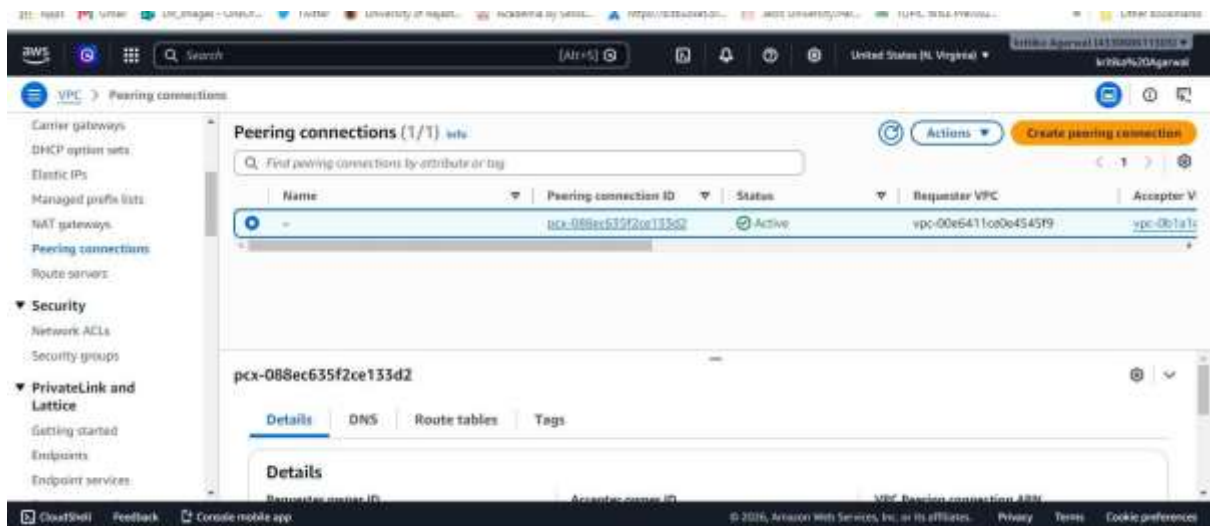


FIG: VPC PEERING (N.VIRGINIA REGION)

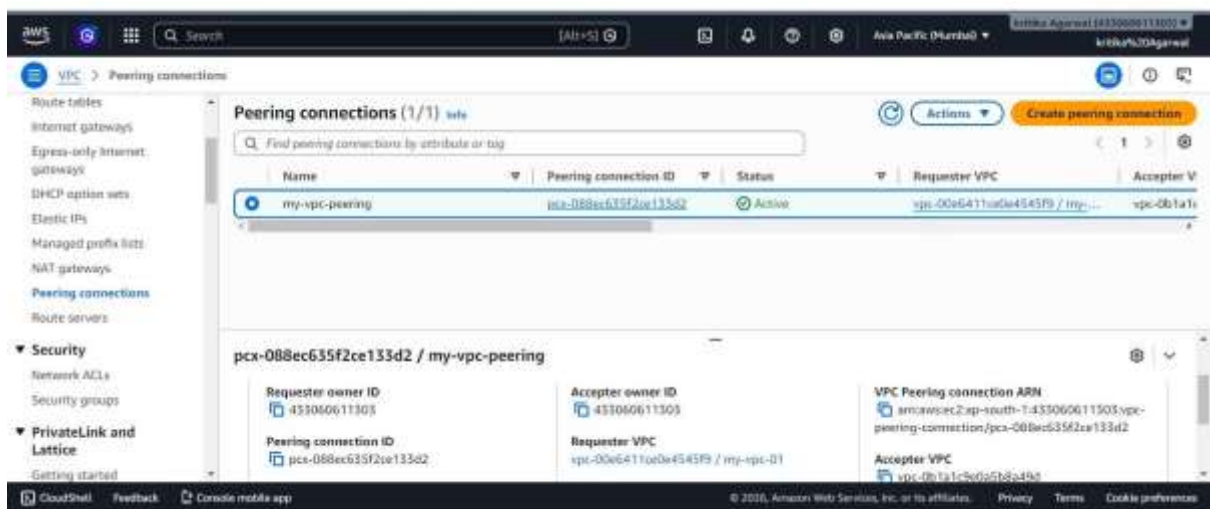


FIG: VPC PEERING (MUMBAI REGION)

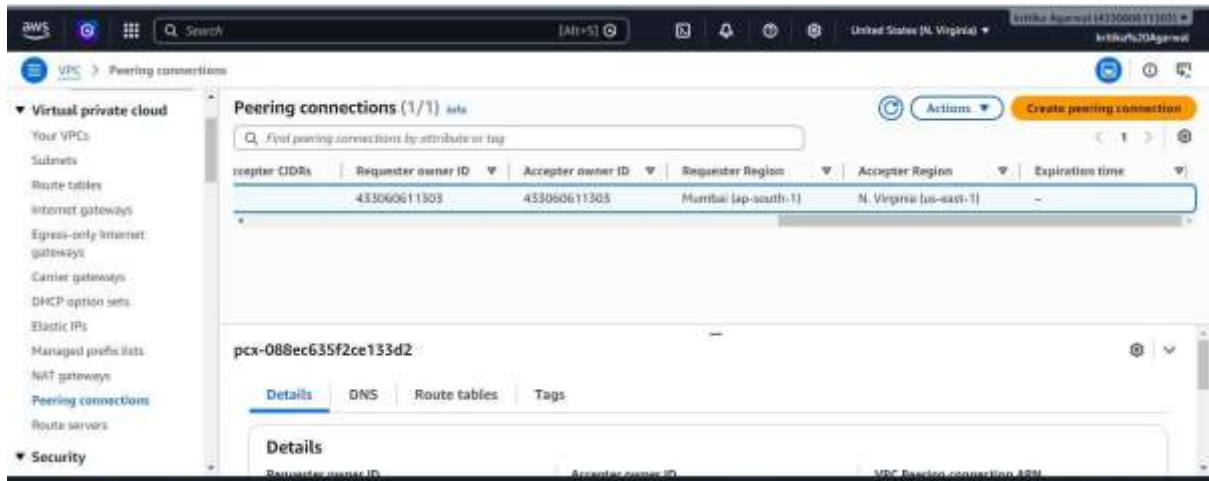


FIG: VPC PEERING IN DIFFERENT REGION

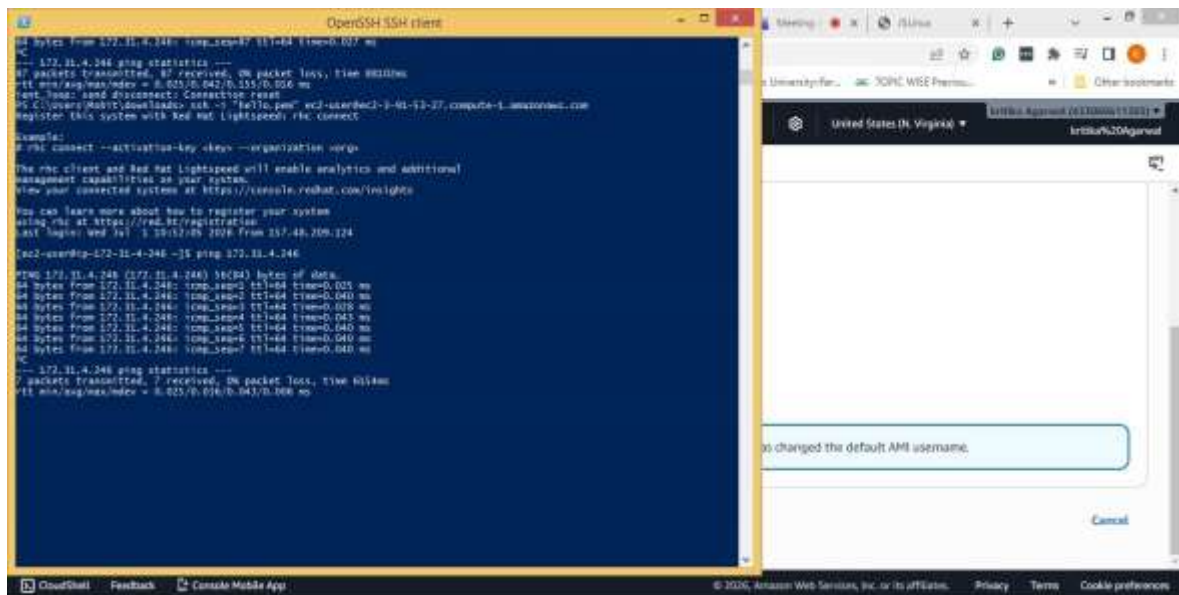


FIG: PING SUCCESSFULLY WORK(PRIVATE)

8. CONCLUSION-

The AWS VPC setup was completed successfully. Public and Private Subnets, Internet Gateway, NAT Gateway, Route Tables, and Security Groups were configured correctly. All resources were tested, and the network was functioning as expected.

The AWS VPC infrastructure was successfully designed and configured with both public and private subnets. Route tables, Internet Gateway, NAT Gateway, and Security Groups were properly configured according to the network requirements.

EC2 instances were launched in the appropriate subnets, and network connectivity was verified. Private communication between EC2 instances was successfully established, and ICMP (ping) testing confirmed that the private network configuration was working correctly.

This project demonstrates a secure and well-structured AWS VPC architecture that enables controlled communication between resources while following networking best practices.

THANK YOU
